

ASP.NET Core CON C#

10. Cifrado de datos sensibles en la bbdd

El cifrado y descifrado de datos es la base principal de la seguridad de los datos que almacenamos en nuestra base de datos, es recomendable cifrar aquellos que sean sensibles. Esto consiste en transformar un texto plano y legible en otro texto codificado. Dentro del cifrado vamos a distinguir 2 tipos

- Cifrado reversible: Permite pasar del texto codificado al original. Lo utilizaremos para cifrar los datos sensibles que no necesitemos proteger en exceso, por ejemplo, un email
- Cifrado no reversible: También llamado hash, este cifrado codifica los datos de manera que no es posible volver al texto original. Lo utilizaremos para cifrar las contraseñas, ya que si partimos del mismo texto y aplicamos el mismo cifrado obtendremos la misma secuencia de caracteres.

Para cifrar la password asignaremos un SALT de contraseña

Salting: Reforzar la seguridad de las contraseñas

Crear una buena contraseña puede hacer mucho para proteger tus datos en caso de que se filtren. Sin embargo, aunque crees una contraseña aleatoria de 20 caracteres con símbolos y números, en algún momento, la seguridad de tus credenciales se te escapa de las manos. Todo depende de cómo almacene tus contraseñas el proveedor de servicios.

Por desgracia, incluso las contraseñas más seguras se pueden descifrar fácilmente si no se protegen de la forma adecuada. Esto es así porque las contraseñas idénticas producen hashes idénticos con las técnicas de hashing tradicionales, lo que, como ya sabemos, las hace vulnerables.

Aquí es donde el salting de contraseñas viene a solucionarlo todo. Básicamente, es una medida de seguridad adicional diseñada para ayudar a proteger tus contraseñas almacenadas contra ciberataques.

Normalmente, tus contraseñas no se guardan en texto sin formato. Antes de almacenarse en la base de datos del sitio web, se someten a un proceso denominado *hashing de las contraseñas*, que transforma las contraseñas en una cadena aleatoria de caracteres con una longitud determinada. Cada vez que accedes a tu cuenta, la contraseña se procesa mediante el mismo algoritmo hash unidireccional. El resultado se compara con los hashes de la base de datos y, si coinciden, se te concede acceso a la cuenta.

Aunque pueda parecer una forma segura de almacenar contraseñas, existe un problema. Si dos contraseñas coinciden, su hash es idéntico, lo que las hace más fáciles de descifrar. Aquí es donde entra en juego el salting o salado de contraseñas. Un salt de contraseña es un bit aleatorio de datos que se añade a la contraseña antes de pasarla por el algoritmo hash. Es diferente para cada usuario, lo que hace que los hashes asignados a cada contraseña también sean únicos.

¿Cómo funciona el salting o salado de contraseñas?

Imagina que tu contraseña es «amarillo». Si otro usuario tiene la misma contraseña, la salida del hash será la misma. Pero si añades unos cuantos caracteres aleatorios a ambas, obtienes dos contraseñas diferentes: «amarillo#1Gn%» y «amarillo9j?L», con hashes completamente diferentes.

¿Qué tipos de ataques se pueden mitigar mediante el salting?

Descifrar una contraseña es básicamente un juego de adivinanzas. Los hackers intentan dar con la combinación de caracteres correcta y, si la contraseña tiene hash, emplean potentes ordenadores para que hagan el trabajo por ellos. Hay tres formas principales de descifrar una contraseña cifrada con hash:

- **Fuerza bruta**

Podría considerarse el método más sencillo para descifrar contraseñas cifradas con hash. Tal y como su nombre indica, se trata simplemente de adivinar todas las combinaciones posibles de contraseñas y luego pasarlas por un programa informático que prueba sus casi infinitas combinaciones. Una vez que obtengas una coincidencia, sabrás la contraseña original. La fuerza bruta es una de las principales razones por las que tus contraseñas deben contener al menos 12 caracteres: los ataques de fuerza bruta suelen funcionar mejor con contraseñas más cortas. Cuanto más largas son las contraseñas, más potencia de cálculo se necesita para descifrarlas.

- **Ataques de diccionario**

[El ataque de diccionario](#) es una versión más sofisticada de la fuerza bruta. En lugar de adivinar al azar, el ordenador prueba con [las palabras y combinaciones de caracteres más comunes en las contraseñas](#). Esta es la razón por la que los ataques de diccionario mejoran con cada filtración de datos: cada vez los delincuentes saben más sobre cómo creamos nuestras contraseñas.

- **Tablas arco iris**

Un ataque de tabla arco iris es una variación de un ataque de fuerza bruta, en el que los hackers emplean tablas arco iris, bases de datos precalculadas de contraseñas hash descifradas, para descifrar los hashes de contraseñas de la base de datos. Los hackers buscan en sus tablas arco iris el hash que necesitan. Al igual que ocurre con los ataques de diccionario, las tablas arco iris tienen más éxito con cada filtración de datos.

Prácticas recomendadas para el salting

Un salt de contraseña sólida es bastante similar a una contraseña fuerte: debe ser única, larga, compleja e imposible de predecir:

1. Que sea aleatoria: nunca utilices las palabras del diccionario o los nombres de los usuarios como salt de contraseña.
2. Haz que sea complicada: combina mayúsculas y minúsculas, números y caracteres especiales como & @ #.
3. Que sea larga: idealmente, la longitud debería ser igual a la salida del hash.
4. Evita los salt reutilizados: un salt de contraseña segura solo se utiliza una vez. Se debe generar una nueva sal cada vez que un usuario cambie su contraseña o cree una nueva cuenta.
5. Mantén los hashes a salvo: almacena los hashes y sus correspondientes salts en un formato seguro y cifrado. Es crucial utilizar medidas de seguridad sólidas para proteger la base de datos donde se almacenan los salts y los hashes para evitar accesos no autorizados.

Errores comunes que hay que evitar al cifrar contraseñas

Es importante tener en cuenta que hay trampas comunes que pueden disminuir la eficacia de esta estrategia. Comprender estos errores es fundamental si quieres aplicar una estrategia de salting que funcione. Estos son algunos de los errores más comunes:

Hashes cortos y predecibles

Aunque el salting añade una capa adicional de seguridad, la fuerza del hash final sigue siendo bastante importante. Si el hash es demasiado corto o simple, debilita la seguridad general del almacenamiento de contraseñas. Un hash corto limita el número de posibles valores hash, lo que a su vez facilita a los hackers el uso de métodos de fuerza bruta para descifrar contraseñas. La misma lógica se aplica a los hashes simples y predecibles. En el salting, asegúrate de que el algoritmo hash que elijas produzca una salida larga y compleja.

Hashes reutilizados

Cada contraseña debe tener su propia sal única. Reutilizar hashes en diferentes registros es lo mismo que utilizar la misma contraseña en varios sitios y servicios. La reutilización de hashes permite a los hackers utilizar técnicas como los ataques de tabla arco iris de una manera muy eficiente.

Base de datos de código abierto

Aunque confiar en una base de datos de código abierto para almacenar contraseñas puede tener sus ventajas, es fundamental darse cuenta de que, sin configuraciones de seguridad adicionales, dicha base de datos podría ser una mina de oro para los ciberdelincuentes. En la mayoría de los casos, los delincuentes están familiarizados con las configuraciones habituales de las bases de datos de código abierto y pueden explicar fácilmente las vulnerabilidades conocidas. Por tanto, si utilizas, o piensas utilizar, una base de datos de código abierto, asegúrate de aplicar medidas de seguridad estrictas, personalizar los ajustes para que no sean los predeterminados y, por último, mantén el software actualizado.

Prácticas recomendadas para la seguridad de las contraseñas

Crear contraseñas seguras y únicas

La base para una buena seguridad de las contraseñas comienza con la creación de contraseñas seguras para cada cuenta. Una contraseña segura debe tener al menos 12 caracteres e incluir una combinación de letras mayúsculas y minúsculas, números y

símbolos. Las contraseñas únicas son importantes porque garantizan que, incluso si una contraseña se ve comprometida, las demás cuentas seguirán estando seguras.

Habilitar la autenticación multifactor en la seguridad de contraseñas

Añade una capa adicional de seguridad al pedir a los usuarios que proporcionen dos o más factores de verificación para acceder a sus cuentas. En la mayoría de los casos, la AMF combina algo que sabes (tus contraseñas) y algo que tienes (tu smartphone, una tablet, etc.), y en algunos casos algo que eres (datos biométricos como la huella dactilar, la cara). La MFA reduce en gran medida el riesgo de acceso no autorizado, incluso si la contraseña se ve comprometida.

Educar a los usuarios sobre la higiene de las contraseñas

La gente debe ser consciente y entender claramente los riesgos asociados a las contraseñas débiles. También deberían darse cuenta de por qué las prácticas recomendadas de seguridad, como no reutilizar contraseñas en distintos sitios y disponer de un gestor de contraseñas para sus datos confidenciales, pueden quitarles un peso de encima en materia de seguridad. Las iniciativas educativas pueden ayudar a reforzar la importancia de una buena higiene de las contraseñas.

Integración del salting en las políticas de seguridad de las organizaciones

El salting debe formar parte de la estrategia de seguridad más amplia de tu empresa. Las organizaciones deben tomar medidas y aplicar políticas que exijan que las sales de las contraseñas sean únicas para cada usuario y garantizar que se almacenen de forma segura.

Utilizar un gestor de contraseñas fiable

Un gestor de contraseñas fiable proporciona un almacenamiento seguro para tus contraseñas, así como para otros datos confidenciales como información personal, tarjetas de crédito y archivos.

Ejemplo: Encriptar y Desencriptar un email

```
public static string EncryptString(string plaintext, string key)
{
    byte[] iv = new byte[16];
    byte[] array;

    using (Aes aes = Aes.Create())
    {
        aes.Key = Encoding.UTF8.GetBytes(key + "2post");
        aes.IV = iv;

        ICryptoTransform encryptor = aes.CreateEncryptor(aes.Key, aes.IV);

        using (MemoryStream memoryStream = new MemoryStream())
        {
            using (CryptoStream cryptoStream = new CryptoStream((Stream)memoryStream, encryptor, CryptoStreamMode.Write))
            {
                using (StreamWriter streamWriter = new StreamWriter((Stream)cryptoStream))
                {
                    streamWriter.Write(plaintext);
                }

                array = memoryStream.ToArray();
            }
        }

        return Convert.ToBase64String(array);
    }
}
```

```
public static string DecryptString(string ciphertext, string key)
{
    byte[] iv = new byte[16];
    byte[] buffer = Convert.FromBase64String(ciphertext);

    using (Aes aes = Aes.Create())
    {
        aes.Key = Encoding.UTF8.GetBytes(key + "2post");
        aes.IV = iv;
        ICryptoTransform decryptor = aes.CreateDecryptor(aes.Key, aes.IV);

        using (MemoryStream memoryStream = new MemoryStream(buffer))
        {
            using (CryptoStream cryptoStream = new CryptoStream((Stream)memoryStream, decryptor, CryptoStreamMode.Read))
            {
                using (StreamReader streamReader = new StreamReader((Stream)cryptoStream))
                {
                    return streamReader.ReadToEnd();
                }
            }
        }
    }
}
```